Adaptive NIFTY Options Trading System

Architecture Specification — Phase 1-10 (v0.5.0)

Generated: 2026-08-16 12:02 UTC

Mode: PAPER (no live orders) | Broker adapter: DhanHQ (broker-neutral interface)

Capital: ₹10,00,000 (configurable) | Strategies: Long Call, Long Put, Debit Spread, NO-TRADE

15 Discord alert types | 3-layer protection | Thesis tracker + MAE/MFE | Ablation testing

1. Objective

Build a fully algorithmic NIFTY options trading system that dynamically determines whether to trade or stay in cash, the current market regime, the appropriate options strategy, directional positioning, expiry and strike selection, entry timing, position size, stop-loss and take-profit, whether an existing position should be held, reduced, adjusted, exited, or reversed, and maximum portfolio and daily risk. The system must optimise for net profitability after brokerage, STT, exchange charges, GST, stamp duty, bid/ask spread and realistic slippage. Tax is reported separately because tax treatment depends on the user's applicable circumstances.

The objective is NOT to maximise the number of trades. The primary objective is to maximise risk-adjusted NET expectancy while controlling drawdown and avoiding unnecessary trades. NO-TRADE is always a valid decision; cash is a position.

2. Core Design Principle

The architecture supports multiple strategies, but strategies are enabled only after statistical validation. Initial strategy families are Long Call, Long Put, and NO-TRADE. Long Straddle, Long Strangle, Debit Spread, Iron Condor, Butterfly, Calendar Spreads and other defined-risk structures remain disabled in config until independently validated via walk-forward backtesting. Forcing a trade is forbidden — if expected net return does not exceed the minimum threshold, the system remains in cash.

3. System Pipeline (Decision Hierarchy)

Every cycle follows this exact order, per spec section 28:

```
NIFTY MARKET DATA
|
v
MARKET REGIME ENGINE ──> VOLATILITY REGIME
|
v
STRATEGY SELECTOR ──> STRATEGY VALIDATION
|
v
OPTION / STRIKE SELECTOR
|
v
RISK ENGINE ──> POSITION SIZING
|
v
EXECUTION (paper | live)
|
v
POSITION MONITOR ──> ADJUST | EXIT | REVERSE
|
v
TRADE JOURNAL ──> PERFORMANCE ANALYTICS
```

If any gate fails, the system emits NO-TRADE for that cycle and the reason is journaled.

4. Technology Architecture

The package is split into clearly separated layers. Broker-specific code is isolated behind the BrokerInterface abstraction so Kite / Angel / Fyers can be added later without rewriting the strategy engine.

```
nifty_engine/
config/ # YAML configs — risk, strategies, broker, trading_hours, costs
models/ # Pydantic schemas — single source of truth for data contracts
data/ # BrokerInterface + DhanBroker + MarketCache + OptionChainBuilder
features/ # Technical (ADX/RSI/EMA/ATR/VWAP) + Volatility + RegimeEngine
strategies/ # StrategyBase + LongCall + LongPut + NoTrade
decision/ # RegimeRunner + StrategySelector + OptionSelector + RiskEngine + PositionManager
execution/ # OrderManager + CostModel + Reconciler + BrokerOrderAdapter
journal/ # TradeLogger + DecisionLogger + PerformanceAnalytics
utils/ # IST time utilities, market open / holiday detection
engine.py # Engine.run_cycle() — single entry point
cli.py # `python -m nifty_engine.cli run --mode paper`
```

5. Market Data Layer

The primary broker/data integration is DhanHQ. The architecture isolates the broker layer behind BrokerInterface so another provider can be added later by implementing the same abstract methods. Required data includes NIFTY spot, OHLC, LTP, volume, VWAP, ATR, ADX, RSI, EMA, market structure, and opening range for the underlying; for the relevant NIFTY strikes: LTP, bid, ask, volume, OI, change in OI, IV, Greeks, expiry, strike, bid/ask spread, and market depth where available; and market context such as India VIX, market breadth, previous-day high/low, previous close, gap, intraday range, and expiry information.

CRITICAL: the data layer NEVER fabricates market data. If the broker returns nothing (missing token, holiday, network error, empty payload), the snapshot is marked data_valid=False and the engine emits NO-TRADE. Today is a market holiday — running the engine produces a clean NO-TRADE log explaining 'market closed'.

MarketCache is an in-memory TTL cache that prevents redundant broker API calls within a single decision cycle. The default TTL is 5 seconds; stale data is re-fetched rather than used.

6. Market Regime Engine

The regime engine classifies the market into discrete states: STRONG_BULL, BULL, WEAK_BULL, NEUTRAL, WEAK_BEAR, BEAR, STRONG_BEAR, RANGE, BREAKOUT, REVERSAL. Classification uses measurable features only — ADX, ADX slope, EMA alignment, VWAP position, price structure, ATR, RSI, relative volume, opening-range breakout, option OI structure, OI change, IV, India VIX, and time of day. All classifications are journaled with reasons so the user can audit WHY a regime was assigned.

Volatility is classified independently from direction because direction does not equal volatility. A bullish market can have low volatility, or explosive volatility. The five volatility states are LOW_VOL, NORMAL_VOL, HIGH_VOL, VOL_EXPANSION, VOL_CONTRACTION. Strategy selection must consider both directional regime and volatility regime; long-premium strategies avoid VOL_EXPANSION environments because IV crush erodes premium even when direction is correct.

7. Strategy Selection Engine

Every enabled strategy is evaluated each cycle. Each strategy produces: expected return, expected loss, probability of success, risk/reward, estimated transaction cost, estimated slippage, IV impact, theta impact, liquidity score, and a confidence score. The selector then calculates expected_net_value = expected_gross_pnl - brokerage - STT - exchange charges - GST - stamp duty - SEBI charges - slippage - spread impact - theta decay. A strategy is selected only if expected_net_value exceeds its configured minimum (default ₹500). Otherwise NO_TRADE wins. Among multiple eligible strategies, the one with the highest expected_net_value (tiebroken by confidence) is chosen.

Mid-day time buckets apply a configurable threshold multiplier (default 1.25x in MIDDAY) to suppress chop-induced false signals.

8. Option Selection Engine

After strategy selection, the engine selects the actual contract. For directional buying it evaluates ATM, 1-strike ITM, and 1-strike OTM candidates. Each option is scored on delta, IV, IV percentile, liquidity, bid/ask spread, volume, OI, OI change, expected move, theta, gamma, premium, and distance from spot. The system does NOT automatically select the cheapest option — the cheapest option may have poor liquidity, high theta, and low probability of profitable movement.

9. Risk Management

Risk-based position sizing is mandatory; sizing based only on available cash is forbidden. The risk engine enforces per-trade loss limits (default 1.5% of capital), daily loss limits (3%), maximum consecutive losses (3 triggers cooldown), maximum trades per day (6), and maximum portfolio exposure (6%). Position size considers premium, stop distance, lot size (75), maximum permitted loss, portfolio exposure, daily loss already incurred, and current drawdown. The system can choose 0, 1, 2, 3, ... lots subject to risk limits. Martingale and

averaging into losing positions are forbidden by config (no_martingale=true, no_averaging_losers=true).

Emergency shutdown triggers include: stale data (30s), abnormal bid/ask spread (>20 NIFTY points), API malfunction, order rejection, broker connectivity issues, unexpected position mismatch, and daily loss limit reached. On any of these the engine halts and emits NO-TRADE for the rest of the session.

10. Transaction Cost Engine

Every backtest and live trade applies these costs. Gross P&L; is NEVER reported as profit; the only metric that matters is NET P&L; AFTER COSTS.

Cost Component	Rate	Applied On
Brokerage	₹20 per order (capped)	Each leg
STT — buy	0.0625% of premium	Buy side
STT — sell	0.0625% of premium	Sell side
Exchange txn charges	0.0448% of premium	Sell side
GST	18% on (brokerage + SEBI + exchange)	Each leg
SEBI charges	₹10 per crore turnover	Each leg
Stamp duty	0.003% of premium	Buy side only
Slippage	1.0 NIFTY point per leg	Each fill
Bid/ask spread impact	1.0 NIFTY point per leg	Each fill

All rates are FY25-26 and live in config/costs.yaml — adjust if the regulator revises them.

11. Position Management

Once a trade exists, the position manager continuously reassesses it and can produce HOLD, ADD, REDUCE, MOVE_STOP, TAKE_PROFIT, EXIT, or REVERSE. Hard rules: stop-loss hits 50% premium loss -> EXIT. Take-profit hits 100% premium gain -> TAKE_PROFIT. Trailing stop moves to entry once position reaches 30% profit (one-way — stops only ever get tighter). Regime flip from bullish to bearish (or vice versa) -> EXIT the opposite-direction position. Adding to a winning position is allowed when the original entry conditions still hold and risk caps permit additional exposure. Adding to a losing position is FORBIDDEN by default.

12. NO-TRADE Decision Paths

The system explicitly supports TRADE / REDUCE / HOLD / EXIT / REVERSE / NO_TRADE. The following conditions trigger NO-TRADE:

- Broker not connected (missing DHAN_ACCESS_TOKEN / DHAN_CLIENT_ID)
- Market closed (holiday, weekend, outside 09:15-15:30 IST)
- Data invalid (stale data, empty option chain, zero LTP)

- Reconciliation failure (abnormal spread, API failure)
- Trading-hours gate blocks new entries (PRE_OPEN, POST_CLOSE, after 15:10 in CLOSING)
- No strategy has positive expected_net_value above its minimum threshold
- Risk engine blocks (daily loss limit, max trades, cooldown, max open positions, zero-lot sizing)
- Option selector cannot find a liquid candidate
- Regime confidence below 0.50

Every NO-TRADE cycle is journaled with the specific reason so the user can audit why the system stayed in cash.

13. Trade Journal & Explainability

Every cycle's decision (including NO-TRADE) is journaled to runs/decisions/decisions.jsonl. Every completed trade is journaled to runs/journal/trades.jsonl. Both files are JSONL (one record per line) for easy ingestion by analytics tools. Each decision record contains: timestamp, action, strategy, regime, volatility regime, confidence, expected_net_value, reasons, and a snapshot summary (spot, VIX, time bucket, chain size). Each trade record additionally contains entry/exit prices, gross P&L;, charges, net P&L;, slippage, holding time, and entry/exit reasons.

Explainability: every decision produces a machine-readable decision explanation block including regime, volatility, strategy, confidence, chosen option, reasons, risk, expected net return, and final action. This block is attached to the Decision object and is printed to console and journaled.

14. Performance Analytics

The performance report aggregates all closed trades and emits: total trades, win rate, gross/net P&L;, average winner/loser, expectancy, profit factor, maximum drawdown, total charges, total slippage, average holding time, and P&L; breakdowns by strategy, by regime, and by volatility regime. The CLI command `python -m nifty_engine.cli report` renders this report to console.

15. Configuration System

All thresholds are externalised to YAML configs under nifty_engine/config/. No thresholds are hard-coded in strategy code — the same code path supports backtest, paper, and live modes; only execution differs.

Config File	Purpose
risk.yaml	Capital, per-trade risk, daily loss, cooldown, emergency shutdown
strategies.yaml	Enable/disable strategies, min expected net value, min confidence, min R/R
broker.yaml	DhanHQ endpoint, NIFTY instrument token, rate limits
trading_hours.yaml	Trading session, holidays, time-bucket thresholds

Config File	Purpose
costs.yaml	Brokerage, STT, exchange charges, GST, SEBI, stamp duty, slippage

16. Development Order + Roadmap to Phase 20

Phases 1-10 are complete. Phase 9 is paused per user request (observe Phase 1-8 in paper for 1 week before live capital). Phase 11+ are planned but not yet built.

Phase	Scope	Status
1	Data ingestion & validation (DhanHQ adapter)	✅ Done
2	Local market-data cache (TTL)	✅ Done
3	Technical + regime + volatility engine	✅ Done
4	Long Call / Long Put + NO-TRADE strategies	✅ Done
5	Risk engine + position manager + paper execution	✅ Done
6	Backtesting engine + cost model integration	✅ Done
7	Walk-forward validation + ablation testing	✅ Done
8	Paper trading + Discord lifecycle alerts (15 types) + thesis-aware 3-layer protection + MAE/MFE	✅ Done
9	Live execution with small capital	⏸ Paused (user chose to observe first)
10	Debit spreads (Bull Call + Bear Put) — first multi-leg strategy	✅ Done
11	Long Straddle / Long Strangle (neutral regime, vol expansion)	🔲 Planned
12	Iron Condor / Butterfly (range regime, premium selling)	🔲 Planned
13	Walk-forward parameter grid search (tune thresholds)	🔲 Planned
14	Live broker reconciliation (order status, fill confirmation)	🔲 Planned
15	Multi-strategy portfolio allocation	🔲 Planned
16	Capital scaling ladder (₹2L → ₹20L)	🔲 Planned
17	Tax reporting (separate STT/exchange/GST breakdown)	🔲 Planned
18	Walk-forward retraining (rolling re-optimisation)	🔲 Planned
19	Alternative data (GIFT NIFTY, USDINR, crude, S&P futures)	🔲 Planned
20	Production hardening (graceful shutdown, monitoring, alerting)	🔲 Planned

17. First Deliverable

This document, the data schema (Pydantic models), the strategy interface (StrategyBase ABC), the regime classification specification (RegimeEngine), the risk engine specification (RiskEngine), the backtesting framework skeleton, the cost model, the configuration system, the test plan, and the initial implementation of Long CE/PE + NO-TRADE — all shipped together. Validation against live data is the next step before adding additional

option strategies.

18. Important Restrictions

- NO martingale — averaging into losing positions is forbidden by config.
- NO forcing trades — cash is the default position when edge is insufficient.
- NO optimisation solely for win rate or total P&L; — expectancy and drawdown matter equally.
- NO ignoring transaction costs or slippage — every backtest and live trade applies the full cost model.
- NO future information in backtesting — walk-forward validation prevents look-ahead bias.
- NO automatic activation of every options strategy — strategies are enabled one at a time after validation.
- NO capital scaling simply because backtest profit increased — scaling requires out-of-sample evidence.
- NO synthetic market data — when real data is unavailable, the system stays in cash.

19. CLI Usage

```
# Run one decision cycle in paper mode
python -m nifty_engine.cli run --mode paper

# Continuous loop every 30 seconds
python -m nifty_engine.cli watch --mode paper --interval 30

# Show risk engine + open positions
python -m nifty_engine.cli status

# Show performance report (win rate, expectancy, drawdown, etc.)
python -m nifty_engine.cli report

# Smoke tests (verify no-trade path)
python tests/test_smoke.py
```

20. Next Steps

- Provide DHAN_ACCESS_TOKEN and DHAN_CLIENT_ID via .env when the market reopens.
- Run `watch` during a live session to capture real decision cycles.
- Build the backtesting engine (Phase 6) using DhanHQ historical candle API.
- Walk-forward validate Long Call / Long Put thresholds before scaling capital.
- Only after positive out-of-sample expectancy, enable Phase 10 (Debit Spreads).